

[← Back to Course](#)

Revision

AIML Interview Preparation

In Progress



Reading Material

⌚ 2 min read

Revision Notes

1.1 NumPy Arrays vs. Python Lists

Attribute	Python List	NumPy Array
Data Types	Heterogeneous (mixed)	Homogeneous (same dtype)
Performance	Slower (interpreted loop)	Faster (C-level vectorized)
Memory	Higher overhead	Compact contiguous memory
Operations	No broadcasting	Supports broadcasting
Use Case	General-purpose	Numerical / matrix computation

Key takeaway: When data is purely numerical and operations are mathematical, NumPy is always preferred over Python lists.

1.2 Dimension and Shape

- **Dimension (ndim):** Number of axes. A 1D array is a vector; a 2D array is a matrix.
- **Shape:** A tuple describing the size along each axis. A (3, 4) array has 3 rows, 4 columns.
- **Reshape:** Changes shape without altering data. Total elements must remain constant.
 - Example: `arr.reshape(2, 6)` on a 12-element array.

1.3 Indexing and Slicing

- 1D: `arr[2]`, `arr[1:5]`, `arr[::-2]` (step)
- 2D: `arr[row, col]`, `arr[0:2, 1:3]` (submatrix)
- **Boolean Indexing:** `arr[arr > 5]` — returns all elements satisfying the condition
- Slices return **views**, not copies — modifications affect the original array.

1.4 Type Conversion

```
arr = np.array([1, 2, 3])
arr_float = arr.astype(float)    # [1.0, 2.0, 3.0]
arr_str = arr.astype(str)        # ['1', '2', '3']
```

Used to prevent dtype mismatch errors in computations.

1.5 Aggregate Functions

Function	Description
<code>np.sum()</code>	Total sum
<code>np.mean()</code>	Arithmetic mean
<code>np.min()</code> / <code>np.max()</code>	Minimum / Maximum
<code>np.std()</code>	Standard deviation
<code>np.median()</code>	Median value

- Use `axis=0` to aggregate **down columns** (per-feature summary).
- Use `axis=1` to aggregate **across rows** (per-sample summary).

The `keepdims` parameter

By default, aggregate functions collapse the axis they operate on, reducing the number of dimensions of the result. Passing `keepdims=True` keeps that axis in the output as a size-1 dimension instead of removing it.

```
arr = np.array([[1, 2, 3], [4, 5, 6]])    # shape (2, 3)

np.sum(arr, axis=1)                        # array([ 6, 15])      shape (2)
np.sum(arr, axis=1, keepdims=True)        # array([[ 6], [15]])   shape (2, 1)
```

Why it matters: The size-1 dimension is what makes the result directly broadcastable against the original array — e.g.

`arr / arr.sum(axis=1, keepdims=True)` divides each row by its own row-sum without needing a manual `.reshape(-1, 1)`.

1.6 Logical Operators, Filtering, and `np.where()`

```
arr = np.array([10, 25, 3, 47, 8])
arr[arr > 20]          # [25, 47]
arr[(arr > 5) & (arr < 30)] # [10, 25, 8]
```

`np.where()` — conditional element selection

```
arr = np.array([15, 8, 23, 4, 42])

np.where(arr > 10, arr, 0)      # [15, 0, 23, 0, 42] — keep value if
np.where(arr > 10, 'H', 'L')    # ['H', 'L', 'H', 'L', 'H']
np.where(arr > 10)              # (array([0, 2, 4]),) — indices where
```

- `np.where(condition, x, y)` — element-wise choice between `x` and `y` based on `condition`.
- `np.where(condition)` (single argument) — returns a tuple of index arrays where the condition is `True`, equivalent to `np.nonzero(condition)`.

Why it matters: Boolean indexing (`arr[arr > 10]`) *extracts* matching elements into a shorter array. `np.where` *preserves the original shape*, substituting non-matching entries instead — the tool of choice whenever downstream code expects the same shape (e.g. capping outliers in place).

1.7 Sorting, Unique Values, and Counting

```
np.sort(arr)          # Returns sorted copy
np.argsort(arr)       # Returns indices that would sort the array
```

`np.unique()` and counting

```
arr = np.array([1, 2, 2, 3, 1, 1, 4])

np.unique(arr)          # [1, 2, 3, 4] — sorted distinct
values, counts = np.unique(arr, return_counts=True)
# values = [1, 2, 3, 4]
# counts  = [3, 2, 1, 1]
```

- `return_counts=True` additionally returns how many times each unique value occurs, aligned index-for-index with the returned unique values.

Why it matters: Common for checking class balance in a labels array or building a quick frequency table without reaching for

`pandas.value_counts()`.

1.8 Matrix Multiplication

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

np.dot(A, B)      # Classic dot product
A @ B             # Preferred operator syntax (Python 3.5+)
np.matmul(A, B)   # Equivalent, clearer for matrices
```

Important: `A * B` is element-wise (Hadamard product), NOT matrix multiplication.

1.9 Broadcasting

Rules applied when operating on arrays of different shapes:

1. If arrays differ in number of dimensions, pad smaller shape with 1s on the left.
2. Arrays with size 1 along a dimension are stretched to match the other.
3. If sizes still don't match after stretching, raise an error.

```
a = np.array([[1,2,3],[4,5,6]]) # shape (2,3)
b = np.array([10, 20, 30])      # shape (3,) → broadcasts to (2,3)
a + b    # [[11,22,33],[14,25,36]]
```

Why it matters: Avoids memory-expensive tiling; fundamental to neural network weight operations.

1.10 Vectorization

- Replacing explicit Python loops with array-level operations.
- NumPy executes vectorized ops in compiled C code — 10x to 100x faster.

```
# Slow
result = [x**2 for x in data]

# Fast (vectorized)
result = np.array(data) ** 2
```

1.11 Array Manipulation

Operation	Function	Use Case
Split	<code>np.split(arr, 3)</code>	Partition into train/val/test
Horizontal split	<code>np.hsplit(arr, 2)</code>	Split column-wise
Vertical split	<code>np.vsplit(arr, 2)</code>	Split row-wise
Stack (new axis)	<code>np.stack([a, b])</code>	Add dimension
Horizontal stack	<code>np.hstack([a, b])</code>	Concatenate column-wise
Vertical stack	<code>np.vstack([a, b])</code>	Concatenate row-wise

[Mark as Complete](#)